

A Dataset details

A key component of any learning-based vulnerability detection technique is having access to high-quality, labeled source code. In this study, we curated a custom dataset from publicly available sources (the OSF repository at <https://osf.io/d45bw/>), which includes real-world C/C++ function-level code examples with vulnerability annotations. The samples collected span multiple vulnerability types such as CWE-119 (buffer overflow), CWE-120 (buffer copy issues), CWE-469 (pointer inconsistency), CWE-476 (NULL pointer dereference), and a category of "CWE-others" covering miscellaneous security flaws. These functions were sourced from large codebases in Debian and GitHub projects, ensuring diversity and realism in the code. The statistical overview of the function length in each class is shown in Figure 1. However, each code sample was rigorously preprocessed to remove irrelevant formatting and comments. We stripped whitespace, indentation, and comment lines while preserving line breaks and code structure so that the input of the model focuses purely on code content and semantics. This preprocessing normalizes the inputs and eliminates noise (e.g., comments or stylistic differences) that is not indicative of vulnerability presence. The raw dataset exhibited notable class imbalance, especially a scarcity of examples for certain vulnerability types (in particular, CWE-469). To address this, we performed an AST-based data augmentation to balance the classes. Using the abstract syntax tree of each function, we applied semantics-preserving transformations such as syntactic noise injection (inserting benign statements or reordering non-critical code), variable renaming (changing identifier names consistently), and dead code insertion (adding no-op code blocks). These augmentation techniques increase the number of samples for under-represented vulnerability classes without altering program logics. After augmentation, each new sample was manually reviewed to ensure that the code still compiles and retains the original program logic and vulnerable behavior. Hence, two domain experts with significant expertise in software vulnerability analysis and two Graduate Research Assistant (GRA) independently examined each generated sample. The GRA provided 20 hours of meticulous validation work for a standard hourly fee of \$25. This expert-led validation process was essential to ensure that the dataset ap-

emnlp2023-latex/fig/draper_st.png

Figure 1: Statistical Overview of Function Length Distribution by Label.

emnlp2023-latex/fig/rcpr.jpg

Figure 2: ROC and PR Curves for Evaluating CodeVul+ Performance.

appropriately depicts real-world vulnerability scenarios, strengthening the validity and relevance of the insight of our study. This process resulted in a balanced dataset across the five categories, as summarized in Table 1, with each vulnerability class and the nonvulnerable category having roughly equal representation. By focusing on C/C++ functions and carefully curating and balancing the data, our dataset is well-suited for training and evaluating vulnerability detectors in a real world environment while mitigating bias from class imbalance.

Class	B.Balance	A.Balance		
	Size	Size	Train size	Test size
CWE-119	1,21,618	4,502	3,605	897
CWE-120	18,750	4,496	3,557	939
CWE-469	1,008	4,500	3,608	892
CWE-476	8,941	4,503	3,632	871
CWE-other	18,117	4,508	3,605	903
Total Set	1,68,434	22,509	18,007	4,502

Table 1: Vulnerability Classes and Data Distribution Before (B. Balance) and After Balancing (A. Balance)

B Benchmark Datasets

We carried out in-depth testing on eight perceived benchmarks that illustrate various distributions and patterns of code vulnerability to evaluate our model’s generalizability and robustness. The collected benchmarks encompass various program-

ming languages, vulnerability types, and application domains which provide a comprehensive assessment framework. We used TF-IDF-based similarity measures [Aizawa \(2003\)](#) to quantify the distributional properties of these datasets and Jensen-Shannon divergence clustering scores [Endres and Schindelin \(2003\)](#), allowing for objective evaluation of the topology feature space and complexity of the dataset. According to Figure 3, the JS divergence scores range from 0.498 to 0.726 indicating different patterns of vulnerability manifestation between codebases. In addition, from Figure 4 the similarity analysis of TF-IDF shows different levels of lexical overlap between datasets, with similarity scores ranging from as low as 0.133 to as high as 0.900, suggesting significant vocabulary commonality. In the following section, a qualitative summary of all datasets utilized in this study is provided, highlighting their characteristics, relevance, and suitability for the research objectives.

ReVeal. The *ReVeal* [Chakraborty et al. \(2021\)](#) dataset is a collection of real-world C/C++ vulnerabilities drawn from security patches in large codebases (Linux Debian kernel and Chromium). Furthermore, both projects have rich, publicly accessible vulnerability reports and reproducibility. This benchmark aligns well with our motivation, as we focus on real-world, evolving codebases, and a diverse domain. The author proposes a roadmap to address the challenges of existing deep learning approaches using representation learning, data deduplication, and class rebalancing techniques. We evaluated the approach using the dataset consisting of 1,014 vulnerable and 1,014 non-vulnerable code samples.

BigVul. *BigVul* [Fan et al. \(2020\)](#) is a large-scale vulnerability dataset for C/C++ extracted from public CVE records. Spanning CVEs from 2002–2019, it links each vulnerability to the corresponding GitHub code changes, providing vulnerable and fixed code segments along with CVE metadata (e.g. CWE types, CVSS scores). Although BigVul contains a broad coverage of real vulnerabilities, it has duplicated samples. To assess the adaptability of the model, we use 585 distinct vulnerable and 585 non-vulnerable samples.

Juliet. The *Juliet* [Boland and Black \(2012\)](#) test suite is a comprehensive synthetic dataset of coded examples in C/C++ (and also available in Java and C) created by NIST. It includes tens of thousands of small programs covering 100+ CWEs (Common

Weakness Enumerations). Each Juliet example is manually crafted with a vulnerable version and a corresponding fixed version, ensuring accurate labels. This dataset offers wide vulnerability pattern coverage and perfect ground truth, albeit in contrived code snippets rather than complex real code. The dataset comprises 1,576 vulnerable and 1,576 non-vulnerable code samples, forming a balanced binary classification corpus in terms of our model evaluation.

Devign. The *Devign* [Zhou et al. \(2019\)](#) dataset consists of manually labeled vulnerable functions collected from four large open-source C projects: Linux kernel, QEMU, Wireshark, and FFmpeg. A team of experts spent 600 hours identifying true vulnerabilities in these codebases, yielding a moderate sized dataset of real vulnerabilities with minimal label error. Unlike auto generated datasets, Devign’s examples are real and highly complex, drawn from various systems code, which was essential for evaluating the GNN-based Devign model. The dataset consists of a balanced set of 852 vulnerable and 1,021 non-vulnerable code samples, curated for binary classification tasks to evaluate our approaches.

MixVul. *MixVul* [Du et al. \(2024\)](#) is a combined benchmark that blends multiple sources of vulnerable code to provide a more diverse training corpus. The author aggregates samples from both synthetic datasets (e.g. Juliet) and real-world datasets (e.g. BigVul, Devign) to form MixVul. The dataset spans multiple programming languages and project types, aiming to balance scale and label accuracy. This mixture brings together the breadth of real vulnerability patterns and the reliability of curated examples, making MixVul a comprehensive testbed with varied contexts. Using the dataset of 1,432 vulnerable and 1,432 non-vulnerable code samples, we evaluated the framework.

DiverseVul. *DiverseVul* [Chen et al. \(2023\)](#) is a recently introduced vulnerable code dataset that significantly expands the diversity of projects. It was built by mining 7,500+ vulnerability-fixing commits across 300+ open-source projects, covering 150 different CWEs. Compared to prior datasets, DiverseVul adds hundreds of new projects and vulnerability types, emphasizing generalization. Its breadth and scale highlight deep learning challenges (e.g. many rare, hard CWEs) and help stress-test models on previously unseen code patterns. We evaluated the approach using the dataset consist-



Figure 3: Clustering Evaluation of 8 Benchmarks Using JS-Divergence.

ing of 766 vulnerable and 766 non-vulnerable code samples.

Vul_fun. The *Vul_fun* Du et al. (2024) dataset is a compilation of vulnerable function examples drawn from real software versions associated with known CVEs. For instance, it includes functions from projects like OpenSSL, Wireshark, QEMU, etc., across various released versions (each function tied to a particular CVE ID). Although smaller in size (only a few thousand functions covering single-digit CVEs), Vul_fun provides multiple vulnerable instances per CVE and their safe counterparts, which is useful for evaluating models’ ability to recognize the same vulnerability in different contexts. The multiversion, multicompiler nature of this dataset simulates a realistic scenario for binary-level analysis and ensures that the model learns consistent vulnerability signatures.

Class	AUC	Acc	Pre	Rec	F1	SP	SN
CWE-119 vs CWE-120	0.824	0.745	0.737	0.758	0.747	0.733	0.758
CWE-120 vs CWE-469	0.978	0.927	0.953	0.901	0.927	0.954	0.901
CWE-469 vs CWE-476	0.988	0.942	0.928	0.954	0.941	0.931	0.954
CWE-476 vs CWE-others	0.858	0.771	0.774	0.791	0.783	0.746	0.791

Table 2: Binary Classification Performance of CodeVul+ Across CWE Class Pairs Using the Draper Dataset.

Draper. The *Draper* Russell et al. (2020) also known as VDISC dataset (Vulnerability Detection in Source Code) is a large-scale C/C++ corpus produced under DARPA’s MUSE project. It consists of ≈ 1.27 million function samples extracted from Debian and GitHub repositories, labeled as vulnerable or not using automated static analysis tools (Clang, Cppcheck, Flawfinder). Although most examples are non-vulnerable (only 2% labeled vulnerable, reflecting 20K+ true positives), Draper’s

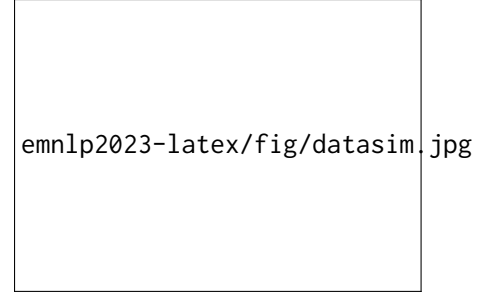


Figure 4: Evaluation of Semantic Similarity Using TF-IDF on 8 Benchmarks.

strength lies in the sheer volume and variety of real code. This dataset is distinctive for evaluating scalability, where its size enables data-hungry training models. Despite the large dataset, we create subset samples as shown in Table 1 for the adaptability analysis between approaches, and the results of the experiments are summarized in Table ?? and Table 3.

Cross language Dataset. The *RealWorld* dataset is utilized to evaluate the performance of adversarial attack evaluations in both our proposed model and the traditional CodePTM framework. We collect clean and adversarially modified datasets which are generated using ChatGPT OpenAI (2023) based on CWE vulnerability patterns. The collected dataset was also thoroughly checked by human annotators and used in adversarial assessment (see Section ??). This verification procedure was carried out in order to facilitate an efficient assessment of the model’s functionality. In addition, the obtained dataset contains equally distributed code samples, where the perturbations were preserved syntactically but semantically altered. To evaluate cross-language generalizability and ensure the inclusion of diverse and representative vulnerability scenarios, we constructed the evaluation dataset using ChatGPT, guided by corresponding CWE vulnerability patterns. This approach enabled the generation of realistic code samples across multiple programming languages, aligned with the objectives of our proposed pretrained style framework. Furthermore, a domain expert and a paid GRA thoroughly reviewed the newly generated dataset to ensure its precision, relevance, and alignment with real-world vulnerability contexts. Afterwards, this new sample used in cross language generalization assessment and the details are described in Section refsec:cross_{gen}).

C C. Classifiers

C.1 Traditional CodePTMs Models

We summarize the characteristics of several model architectures, which can be used for code vulnerability classification by treating code as a sequence (like natural language) or leveraging code structure, as well as a basic tabular representation for classify.

- **CodeBERT Feng et al. (2020)**: A Transformer-based encoder pre-trained on source code and natural language (NL) pairs. CodeBERT learns a joint embedding for code (in languages such as Python, Java, C, C++) and comments/docstrings, enabling it to capture lexical and syntactic features of input tokens. When fine-tuned for vulnerability detection, CodeBERT essentially treats code as a linear sequence of tokens (much like a NL sentence) and excels at understanding identifier names and API usage patterns. Its strength is in semantic similarity and pattern recognition from large-scale pre-training, though it may overlook deep structural context (as provided similar code in Figure ??) since it doesn't explicitly model data/control flow.

- **GraphCodeBERT Guo et al. (2021)**: An extension of CodeBERT that incorporates code structure by leveraging data flow graphs during pre-training. GraphCodeBERT's encoder is transformer-based, but was trained on code with an additional data flow signal, allowing it to better represent relationships between variables and statements. In vulnerability detection, this translates to improved awareness of how data moves through the code (e.g., tainted variables), which can highlight vulnerabilities involving certain dataflow patterns (like use-after-free or uninitialized data flows). GraphCodeBERT thus combines sequence understanding with graph semantics, often achieving higher accuracy on tasks where structural context is crucial. However, this primarily learns from static token sequences and relies on predefined abstract syntax structures without utilizing task-specific structural signals. This creates bottleneck drawbacks for better adaptation to task-specific code semantics.

- **UniXcoder Guo et al. (2022)**: A unified cross-modal pre-trained model for code that can handle code understanding and generation tasks. UniXcoder is based on a transformer with dual-mode training: it can function as an encoder (for understanding tasks like classification) or a decoder (for code generation), and it was trained on code in multiple programming languages with NL com-

ments. For classification, UniXcoder encodes code snippets similarly to CodeBERT, but its training objectives (which include denoising and cross-modal generation) endow it with robust representations even for incomplete code. In our context, UniXcoder serves as a strong encoder for code, with the flexibility to attend to both left and right context (bidirectional) and produce a rich vector representation. Its "unified" nature means it can leverage representations learned from docstring prediction, code translation, etc., potentially giving it an edge in capturing intent and semantics that correlate with vulnerabilities. However, this framework's limitation lies in its lack of structural adaptability, as UniXcoder mainly processes code as a sequence of tokens without explicitly modeling the dynamic control or data flow between them.

- **CodeT5 Wang et al. (2021)**: is a Transformer-based encoder-decoder model pre-trained on large code corpora with text tasks (e.g. code summarization, generation). For classification, we utilize CodeT5's encoder to obtain a sequence representation of the input code. Like CodeBERT, CodeT5 treats the code as a linear token sequence; it excels at capturing token-level semantics and learned patterns from its multi-task pre-training. In our paper, fine-tuning CodeT5 for vulnerability detection allows it to use its rich syntactic and semantic knowledge of code tokens. However, since CodeT5 does not explicitly incorporate program structure (it was not trained with code graphs), it may miss deeper dependency relationships, focusing mainly on sequence information.

- **SantaCoder Ben Allal et al. (2023)**: SantaCoder is a pre-trained transformer model of the 1.1B parameter which is pretrained on multiple programming languages (as part of the BigCode project). It is an autoregressive language model for code, originally aimed at code completion and generation. We adapt SantaCoder for classification by feeding the code and using the contextual embeddings of the transformer (for example, using the final hidden state of a special classification token or averaged token embeddings) to predict vulnerability. SantaCoder brings extensive knowledge of code syntax and common patterns learned from its broad pre-training. It can recognize various coding styles and libraries. However, being a generative model, SantaCoder is not specialized in vulnerability features; it treats code as a simple sequence of tokens, without explicitly capturing data flows

Classifiers	AUC	Acc	Pre	Rec	F1	MCC	K	MSE	MAE	SP	SN	FP	MK
DeepWalk (Perozzi et al., 2014)	0.761	0.439	0.443	0.438	0.439	0.301	0.299	3.291	1.221	0.861	0.438	2529	0.303
LSA (Landauer et al., 1998)	0.791	0.483	0.501	0.484	0.497	0.355	0.354	3.297	1.188	0.880	0.484	2295	0.009
GloVe (Pennington et al., 2014)	0.811	0.515	0.539	0.514	0.521	0.395	0.395	3.289	1.178	0.880	0.484	2215	0.682
FastText (Chen et al., 2022)	0.816	0.537	0.541	0.536	0.537	0.422	0.421	3.096	1.081	0.884	0.536	2081	0.425
FT-Transformer (Gorishniy et al., 2021)	0.778	0.478	0.474	0.479	0.461	0.355	0.349	3.272	1.192	0.478	0.478	2346	-0.004
TabNet (Arik and Pfister, 2021)	0.789	0.507	0.507	0.506	0.504	0.384	0.384	1.159	3.294	0.506	0.506	2115	0.001
SAINT (Somepalli et al., 2021)	0.773	0.473	0.465	0.471	0.453	0.345	0.340	1.197	3.285	0.471	0.471	2370	-0.006
Tabular Perceptron (Brunebarbe, 2022)	0.671	0.417	0.403	0.415	0.384	0.281	0.271	3.002	1.201	0.415	0.415	2623	-0.012
AutoInt (Song et al., 2019)	0.794	0.485	0.509	0.484	0.484	0.362	0.357	3.174	1.164	0.484	0.484	2315	0.002
TabTransformer (Huang et al., 2020)	0.791	0.503	0.526	0.503	0.501	0.382	0.379	3.201	1.148	0.875	0.503	2234	0.701
LSTM (Alimam et al., 2024)	0.816	0.526	0.563	0.526	0.532	0.412	0.401	3.101	1.102	0.881	0.526	2129	0.722
LSTM Gru (Yu et al., 2021)	0.821	0.531	0.421	0.531	0.468	0.431	0.414	2.844	1.041	0.883	0.531	2114	0.657
NN (Alimam et al., 2024)	0.855	0.594	0.601	0.594	0.595	0.493	0.493	2.791	0.964	0.898	0.594	1825	0.749
CodeVul+ (This study)	0.896	0.671	0.669	0.661	0.663	0.577	0.576	2.469	0.831	0.915	0.661	1524	0.339

Table 3: Comprehensive Multiclass Prediction Performances Obtained from Different Models

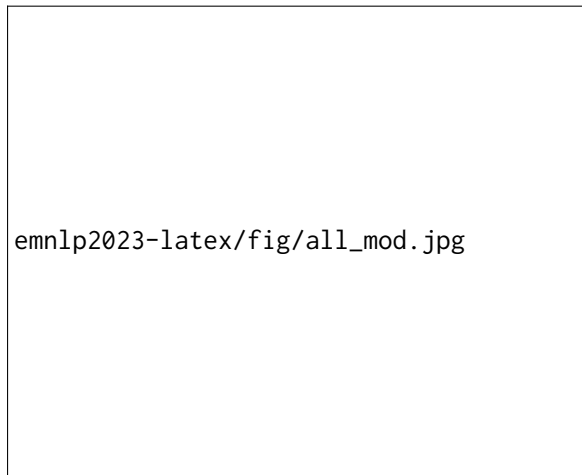


Figure 5: Model Wise ROC Scores for Multiclass Vulnerability Detection.

or underlying graph structures and this may lead to similar treatment of semantically distinct code, potentially overlooking essential contextual differences. However, in our benchmarks assessments, SantaCoder serves as a strong LLM baseline, showing a robust understanding of code semantics.

C.2 Traditional NLP-Based Models

We evaluated the performance of several traditional and pre-trained NLP models in multiclass vulnerability classification, including GloVe, FastText, LSA, DeepWalk and BERT. These models offer strengths in language representation: BERT grasps contextual semantics through deep bidirectional transformers, DeepWalk leverages random walks for structural graph encoding, LSA and GloVe model latent semantic spaces, and FastText incorporates subword-level information for robustness to rare tokens. However, their performance in code specific multiclass vulnerability tasks remains sub-optimal, due to lackings of semantic and structural mismatch between natural language and program-

ming code. As for code, it requires hierarchical, syntactic, and control-flow-aware modeling beyond shallow token co-occurrences. These drawbacks highlight the importance of hybrid strategies like ours, where we incorporate code-aware models with graph-based reasoning for deeper code understanding.

C.3 Traditional Transformer Models

We also evaluated predictive performance on a variety of tabular-based deep learning architectures, including FT-Transformer, TabNet, SAINT, Tabular Perceiver, TabTransformer, and AutoInt. These models have demonstrated potential in a variety of structured data tasks and are particularly excellent at capturing feature interactions Huang et al. (2020). However, when used with source code, they performed considerably lower in code-specific architectures, as shown in Table S3. This drop stems from their incapable capacity to model syntactic hierarchies, execution flows, lack understanding of token-level granularity and inter-line relationships that are essential to code semantics.

C.4 Custom Classifier Head

The central novelty of this study lies in the embedding framework, a fusion of pretrained transformer embeddings with graph reasoning. However, for the vulnerability classification head on top of our embedding framework, we used stacking classifier architecture, which conveys a non-trivial significant advancement. In terms of ensemble classification, the head is formed with SVM Grønlund et al. (2020), XGBoost Ma et al. (2023), Extra Trees Pisane et al. (2009), Logistic Regression (LR) Schein and Ungar (2007), and a unified neural decision layer Moon et al. (2021). This approach improves both robustness and decision boundary sen-

emnlp2023-latex/fig/classifer_abla.png

Figure 6: BaselineModels Wise Classification Results of Multiclass Vulnerability Detection.

sitivity in multiclass tasks where SVM contributes margin-based generalization, XGBoost captures intricate non-linear patterns, LR maintain high bias and low variance nature of data, and Extra Trees provide resilience to feature noise and outliers. In contrast to traditional softmax heads that just use the final transformer embedding, where our classifier model leverages gained feature reweighting and calibrated decision fusion by concatenating their probability distributions and passing them through a simple neural classifier. Empirical evaluations generally indicated that this architecture results in significant performance gains (Table ??,3), particularly in adversarial contexts (Table ??), and provides a generalization for fine-grained multiclass vulnerability classification. In Figure 6 we observed that the stacked model significantly outperforms all base learners reflecting improved generalization for multiclass implies its effective integration of multiple decision boundaries. The baseline model performance was obtained following the systematic hyperparameter tuning process and the configurations of which are summarized in Table 4.

Model	Search Space	Best Parameters
Logistic Regression	C: [0.001, 0.01, 0.1, 1, 10, 100] penalty: ['l1', 'l2'] solver: ['liblinear', 'saga']	C = 10 penalty = 'l2' solver = 'liblinear'
SVM	C: [0.01, 0.1, 1, 10, 100] gamma: ['scale', 'auto', 0.001, 0.01, 0.1, 1] kernel: ['rbf', 'poly', 'sigmoid']	C = 10 gamma = 0.1 kernel = 'rbf'
XGBoost	max_depth: [3, 4, 5] learning_rate: [0.01, 0.1, 0.2] n_estimators: [100, 200, 300] subsample: [0.8, 0.9, 1.0] colsample_bytree: [0.8, 0.9, 1.0]	max_depth = 4 learning_rate = 0.2 n_estimators = 100 subsample = 1.0 colsample_bytree = 0.9
Extra Trees	n_estimators: [50, 100, 200] max_depth: [None, 10, 20, 30] min_samples_split: [2, 5, 10] min_samples_leaf: [1, 2, 4] criterion: ['gini', 'entropy']	n_estimators = 200 max_depth = 30 min_samples_split = 5 min_samples_leaf = 2 criterion = 'gini'

Table 4: Search space and best hyperparameters for each model.

C.5 SOTA Models

In this following section, we describe the prior vulnerability detection models which we compare against, highlighting their architecture, the datasets they were evaluated on originally, and how they relate to our approach:

- **Devign Zhou et al. (2019)**: Devign introduced a graph neural network (GNN) approach for vulnerability detection, treating each function as a graph of code elements. Its architecture uses a gated graph neural network (GGNN) operating on a code property graph that combines an abstract syntax tree, control flow and data flow information. A novel convolution module in Devign’s GNN aggregates node representations to produce a function-level vulnerability prediction. Devign was originally trained and tested on their hand-labeled dataset of C functions from Linux, QEMU, FFmpeg, and Wireshark (the “Devign dataset”), and achieved state-of-the-art results at the time, significantly outperforming the model based on patterns and sequences. Devign’s composite code graph embeddings for binary vulnerability detection are innovative; however, its design has significant drawbacks when used to multiclass situations. After carefully re-implementing Devign, we observed that its Gated Graph Recurrent Layers and Conv module, which are optimized for binary cases but fall short to capture fine-grained class separability among multiple vulnerability categories. From Table ?? this underperformance is evident in multiclass evaluations that is limited to semantic abstraction, lack of contextualized token-level reasoning, and insufficient label diversity during representation learning. In contrast, our proposed CodeVul+ leverages pretrained LLM style embeddings integrated with multilayer semantic reasoning, enabling it to retain hierarchical code context and generalize across classes. The result obtained from CodeVul+ substantiates the importance of modular pretraining, deeper graph inference, and task aligned fine tuning for scalable vulnerability classification.

- **IVDetect Li et al. (2023a)**: IVDetect is an interpretable deep learning model that not only predicts vulnerabilities but also pinpoints vulnerable code portions. It uses a program dependency graph (PDG) to separate a function into vulnerable statements and their surrounding context. Essentially, IVDetect’s architecture encodes the potentially vulnerable lines (e.g., a call to a risky API) and the con-

control/data dependencies around them in two streams, to better focus on the vulnerability-relevant subgraph rather than treating the function monolithically. Then it provides fine-grained interpretations by highlighting the PDG subgraph corresponding to the detected vulnerability. Although IVDetect has a strong fine-grained structure, our replication of the structure for multiclass vulnerability classification yielded suboptimal performance. From Table ?? the result obtained by IVDetect indicates the struggle to generalize across multiple vulnerability classes. In contrast, our framework consists of dynamic hybrid graph reasoning, enabling it to capture a richer spectrum of vulnerability semantics and contextual cues across diverse classes and achieve significantly higher predictive results.

- **ReVeal Chakraborty et al. (2021)**: ReVeal is a recent model designed specifically to handle the real-world complexities dataset. It consists of three main modules: a gated GNN (for code structure learning), a SMOTE-based oversampling component (to address class imbalance), and a triplet loss-based separation learning block that combines these features. ReVeal’s GNN processes the code property graph of a function, similar to Devign. The authors showed that many prior models (including Devign and others) struggled on the ReVeal dataset, achieving misleadingly high scores on easier benchmarks but failing on others’ complex data. When rebuilding this architecture for multiclass prediction tasks, we observed a significant performance drop shown in Table ??, indicating limitations in its adaptability. Such as a fixed graph-based representation is inflexible in capturing control flow, syntactic variants, or token-level semantics, which is crucial for multiclass generalization. However, our CodeVul+ builds on a similar GCN backbone idea, but we extend it with a richer input through multi layer graph reasoning, aiming for a more generalizable solution.

- **MANDO Nguyen et al. (2023)**: MANDO is a deep learning framework originally proposed for smart contract (Solidity) vulnerability detection, which we include as a baseline to test cross-domain robustness. It uses multi-level heterogeneous graph embeddings, which constructs a graph that includes contract-level nodes, function-level nodes, and code-level elements, capturing the hierarchical structure of smart contracts. MANDO was evaluated on Ethereum smart contract datasets (e.g., SolidiFI and others) and outperformed earlier con-

tract vulnerability detectors by learning from both source code and contract-level context. According to Table ?? predictive results, this strong approach falls short in terms of multiclass task where its major limitation is its tight coupling to Solidity specific code structures and semantics, which limits generalizability. MANDO relies on call graph and static control-flow representations extracted through tools like Slither, reduces its practical adaptability to other programming languages or more diverse code corpora. In contrast, our CodeVul+ approach addresses these limitations by precisely capturing the intricate local and global code context. In general, CodeVul+ not only generalizes better, but also captures the semantic-structural duality of code more effectively.

BGNN4VD Cao et al. (2021): Bidirectional Graph Neural Network for Vulnerability Detection is a GNN-based approach that explicitly constructs code graphs and propagates information in both forward and backward directions. BGNN4VD’s pipeline first extracts multiple graph representations of the code: AST, CFG, and DFG fused into a Code Composite Graph (CCG) for each function. In the learning phase, they used bidirectional GNNs and CNN modules for classification. The bidirectional GNNs effectively captured information flow by the advantages of the influence of both predecessor and successor nodes in the graph structure. However, we reproduce their architecture for multiclass vulnerability prediction, we noticed a significant gap in performance even though we carefully followed the procedures outlined in their paper. Their dependences on bidirectional GNNs, CNN modules and Word2Vec for code token representation are limited in capturing deep, contextual relationships between code elements, which could be the key constraints.

ContraFlow Cheng et al. (2022): is a recent approach that focuses on learning path-sensitive embeddings for code using contrastive learning. The core idea of ContraFlow is to identify crucial value-flow paths in the program that are likely relevant to vulnerabilities. Several augmented views of code forms are used in this approach, including ASTs, CFGs, and data flow graphs, which helps to capture structural semantics. View-aware projection heads are also used with contrastive learning to maintain consistency between heterogeneous graph encodings during representation. As they don’t share the code repositories, we carefully reimple-

mented this architecture following its described pipeline and adapting it for multiclass classification. However, this optimized approach performed significantly poorly for the multiclass vulnerability task. Although the method effectively integrates contrastive learning across various code representations, it has a limited ability to capture subtle structural and semantic variations that are essential to distinguish between different vulnerability classes. In contrast, our proposed CodeVul+ framework addresses this gap through a pretraining-style modular that helps to grasp class-sensitive feature representations.

μ VulDeePecker Zou et al. (2020): is extends the original VulDeePecker framework by introducing a deep learning architecture designed for multiclass vulnerability detection. It leverages both data-dependence and control-dependence relationships and employs a BiLSTM network enhanced with an attention mechanism to effectively integrate global and local contextual features. Despite using BLSTM networks in its architecture to combine global and local syntactic representation features, it is unable to fully capture structural variances and hierarchical semantic connections across various vulnerability classes. As a result, when we carefully reimplemented the architecture following the description and evaluated it on our multiclass benchmark, the performance remained significantly low, indicating gaps in representation robustness and generalization. In contrast, our framework addresses these gaps by dynamic constructed graphs reasoning and transformer derived embeddings improved utilization of contextual and structural cues.

Projects	AUC	Acc	Pre.	Rec.	F1	MCC	K.	MSE	MAE	SP	SN
Python	0.566	0.583	0.575	0.633	0.601	0.167	0.167	0.416	0.416	0.533	0.633
JavaScript	0.591	0.551	0.533	0.801	0.641	0.115	0.101	0.451	0.452	0.321	0.801
Java	0.437	0.451	0.463	0.633	0.535	-0.107	-0.101	0.551	0.551	0.311	0.631
C	0.714	0.651	0.621	0.766	0.686	0.308	0.301	0.351	0.351	0.534	0.766
C++	0.547	0.466	0.471	0.533	0.501	-0.067	-0.067	0.532	0.522	0.401	0.533
Go	0.488	0.451	0.457	0.533	0.492	-0.101	-0.101	0.551	0.551	0.366	0.533
PHP	0.561	0.533	0.527	0.631	0.575	0.067	0.063	0.466	0.466	0.433	0.633
Ruby	0.501	0.434	0.428	0.401	0.413	-0.133	-0.131	0.566	0.566	0.467	0.401
Rust	0.522	0.466	0.472	0.566	0.515	-0.068	-0.066	0.532	0.533	0.367	0.566

Table 5: Performance Comparison of Binary Classification Metrics Across Programming Language Projects Using the Proposed *CodeVul+* Model in the Domain of Code Maintainability and Refactoring

SySeVR Li et al. (2023b): stands for Syntax-based and Semantics-based Vector Representation, a framework for vulnerability detection using deep learning that emphasizes enriched program representations. SySeVR is built on the insight that neither purely syntax-based nor purely semantic-based code models are sufficient alone; thus, it con-

structs program vectors that capture both aspects. The SySeVR pipeline involves extracting program slices focused on potential vulnerabilities and generates multiple representations for each code slice: one that is syntax-oriented while another one is semantics-oriented. These representations are then encoded, often by an RNN or other neural network, into vectors. When comparing SySeVR to CodeVul+, a few distinctions emerge: (1) SySeVR relies on a predefined static analysis step to extract the likely vulnerable slices (pattern-based), whereas CodeVul+ examines the entire function with no pre-filtering (the model itself learns to attend to the important parts, with the help of GCN and interpretability module). (2) SySeVR’s representations are hand-engineered to an extent (separating syntax and semantics), while CodeVul+ uses a more unified neural representation. (3) In terms of training, SySeVR had to train an embedding model from scratch on its vulnerability dataset, whereas CodeVul+ benefits from transfer learning. Indeed, in our results, SySeVR performs relatively well on known datasets but CodeVul+ is outperformed in most cases. CodeVul+ achieved higher accuracy and robustness, particularly in cross-project evaluations, likely because its pre-trained knowledge and graph reasoning can capture vulnerability patterns that SySeVR’s fixed slicing might overlook.

D Algorithm

In our proposed framework, we integrated structural graph representation with pre-trained code embeddings, enabling the capability of both semantic and structural code properties. We used Laplacian matrices transformation from graphs and token embeddings to address a significant limitation in existing methods that often miss vulnerabilities dependent on distant but logically connected code segments. Additionally, this normalized Laplacian transformation facilitates effective message passing across the graph structure, enabling the GCN’s to learn vulnerability relevant structural patterns. In contrast, the consistently low validation loss across epochs that is shown in Figure 7 underscores the robustness and generalizability of the framework, strengthening the dependability and caliber. Similarly, this produces a feature space with better ability to discriminate between vulnerabilities by optimizing the original embeddings through structural reasoning. The general code vulnerability detection methodology is depicted in Algorithm 1.

emnlp2023-latex/fig/tr-val.jpg

Figure 7: Training and Validation loss Curves of the proposed model. The consistent downward trends without divergence between training and validation indicate stable convergence and an efficient learning.

E Evaluation Metrics

To thoroughly evaluate the performance of the model, we present a set of metrics that capture different aspects of binary and multiclass classification. The overall effectiveness of vulnerability detection models is assessed using metrics such as AUC, Accuracy, Precision, Recall, and F1, which highlight the trade-offs between false negatives and false positives across various vulnerability classes. In security-critical domains where balanced performance is paramount, these comprehensive measurements are essential. Additionally, we used Matthews Correlation Coefficient (MCC), Cohen’s Kappa, Mean Squared Error (MSE), Mean Absolute Error (MAE), Specificity (SP), Sensitivity (SN), and Markedness (MK) to quantify subtler aspects of model performance compared to traditional metrics, providing a more reliable evaluation on skewed vulnerability datasets [Chicco and Jurman \(2022\)](#).

Matthews Correlation Coefficient (MCC): MCC is a correlation coefficient between predicted and actual binary labels, often regarded as a balanced measure even if classes are imbalanced.

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Cohen’s Kappa (Kappa): Cohen’s κ measures agreement between the predictions of the model and the ground truth, adjusted for chance agreement.

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where;

$$p_o = \frac{TP + TN}{N}$$

$$p_e = \frac{(TP + FP)(TP + FN) + (TN + FN)(TN + FP)}{N^2}$$

$$N = TP + TN + FP + FN$$

Mean Squared Error (MSE): MSE is computed between the predicted score $\hat{y}$ and the actual label y (1 for vulnerable, 0 for not).

$$MSE = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Mean Absolute Error (MAE): MAE is the average absolute difference between the predicted score and the actual label.

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

Markedness (MK): Measures the trustworthiness of predictions [Ingram et al. \(2016\)](#).

$$\text{Markedness} = \text{Precision} + \text{NPV} - 1$$

Where NPV (Negative Predictive Value):

$$NPV = \frac{TN}{TN + FN}$$

Algorithm 1 End-to-End Pipeline of this Study

```

1: Input: Training set  $C_{train}$ , target set  $C_{target}$ ,
   classifier  $F$ , pretrained embedding model
    $E_{bert}$ 
2: Output: Vulnerability predictions with high-
   lighted vulnerable lines
3: Initialize hybrid model  $H_{gcn}$  with embedding
   dim=768, hidden=512, out=768
4: Compute optimal sequence length  $L$  from to-
   ken distribution in  $C_{train}$ 
5: Initialize  $E_{train} \leftarrow \emptyset$ ;  $E_{target} \leftarrow \emptyset$ 
   BuildGraph  $c$ ,  $sequence\_length$ 
6:  $G \leftarrow build\_graph(sequence\_length, 0.2)$ 
7:  $A \leftarrow adjacency\_matrix(G)$ 
8:  $L \leftarrow normalized\_laplacian(A)$ 
9: return  $L$ 
10: for  $epoch = 1$  to  $N_{epochs}$  do
11:   for each batch  $B \subset C_{train}$  do
12:      $X_{emb} \leftarrow E_{bert}(tokenize(B, L))$ 
13:      $G_{lap} \leftarrow \{BuildGraph(c, X_{emb}.shape[1]) | c \in B\}$ 
14:      $Y_{gcn} \leftarrow H_{gcn}(X_{emb}, G_{lap})$ 
15:     Update  $H_{gcn}$  by minimizing
        $\mathcal{L}(mean(Y_{gcn}), mean(X_{emb}))$ 
16:   end for
17: end for
18: Save  $H_{gcn}$  for downstream reuse
19: for each batch  $B \subset C_{target}$  do
20:    $X_{emb} \leftarrow E_{bert}(tokenize(B, L))$ 
21:    $G_{lap} \leftarrow \{BuildGraph(c, X_{emb}.shape[1]) | c \in B\}$ 
22:    $features \leftarrow mean(H_{gcn}(X_{emb}, G_{lap}))$ 
23:    $E_{target} \leftarrow E_{target} \cup \{features\}$ 
24: end for
25: Train classifier  $F$  on  $E_{target}$ , get predictions
    $Y$ 
26: for each  $(c, y) \in (C_{target}, Y)$  do
27:    $lines \leftarrow LIME(c, y, F)$  {Highlight
     weighted influential code lines}
28: end for
29: return  $Y$ , highlighted  $lines = 0$ 

```

F Implementation and Hyperparameters

Implementation Details. In our approach, we leverage a carefully optimized setup that offers strong predictive performance while ensuring efficiency in both training and inference. For the purpose of capturing rich syntactic and semantic structures from source code, we utilize PyTorch

(for neural network building blocks) and Hugging-Face Transformers as the backbone of our CodeVul+. As for the pretrained transformer, we incorporate GraphCodeBERT initialized with pretrained weights to leverage its strong syntactic and semantic code understanding capabilities. The maximum sequence length is dynamically determined on the basis of the 95th percentile of token lengths in order to reduce extraneous padding and maximize computing effectiveness. In addition on top of the Transformer embeddings, we integrate a graph reasoning module composed of two sequential GCN layers and layer normalization is then used to stabilize the feature distribution. As for dynamic graph constructions, we use Erdős-Rényi graph topology and laplacian normalization then used for creation of adjacency metrics from the graph, which facilitates relational structure modeling between code tokens in an adaptive manner without the need for explicit AST or CFG processing.

Hyperparameters. We use hyperparameters designed to balance learning stability for model optimization, generalization, and convergence speed for real-world vulnerability detection tasks. The parameters used for CodeVul+ are as follows:

- **Learning Rate:** Set to $1e-4$, a conservative value on top of the pretrained GraphCodeBERT encoder that stabilizes fine-tuning, that avoids catastrophic forgetting, and permits the graph reasoning module to converge smoothly.
- **Batch Size:** The value was set to 8 to optimize GPU consumption while preserving steady training patterns and preventing gradient noise, particularly important given the variable length structure of code snippets.
- **Optimizer:** We used Adam owing to its superior convergence behavior and adjustable learning rate capabilities in complex, high dimensional spaces that are frequent in code representation learning.
- **Loss Function:** To match transformer feature representations with graph reasoned embeddings, we employ Mean Squared Error (MSE) loss, ensuring that significant semantic changes are captured by the model without imposing hasty classification decisions.
- **Graph Construction:** An edge connection probability of 0.2 is used for the dynami-

735
736
737
738

739
740
741
742
743
744

745
746
747
748
749
750
751
752

753

754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783

cal graph construction in the Erdős–Rényi graph, keeping node interactions from becoming overly saturated while encouraging moderate connectivity.

- *Sequence Length*: We set the maximum sequence length dynamically to the 95th percentile of word lengths in the dataset rather than setting it at a fixed number, reducing unnecessary padding and increasing computing effectiveness in both inference and training.

All baseline models were re-implemented or obtained from the open source code of the authors and trained in our environment for a fair comparison. We ensured that crucial hyperparameters including learning rate, architecture design, and training configurations were carefully aligned with those proposed in the original article introducing the baseline approach.

G Training and Inference Efficiency

On a high-performance machine, the experimental assessments were performed featuring an integrated NVIDIA RTX 4080 GPU setup with 12 GB of dedicated graphics memory. The implementation used the core deep learning pipeline and the PyTorch for deep learning framework was chosen for model implementation because of its flexibility and speed in model creation and training. Additionally, we used a variety of evaluation metrics to capture both global and class-specific behavior, including accuracy, precision, recall, f1-score, MCC, and Cohen’s Kappa. To further ensure models’ robustness, we used regression-based error measures (MSE, MAE) and statistical indicators (Specificity, Sensitivity, False Positive Rate and Markedness). Throughout the study, this combination of metrics and resources made it easier to experiment and evaluate the model. Our proposed CodeVul+ framework exhibits very effective inference and training features, primarily owing to the modular hybrid architecture that incorporates lightweight graph-based reasoning layers with pretrained transformer embeddings. Our model rapidly converges in just three epochs, achieving a smooth and consistent reduction in both training and validation loss, as shown in Figure 7. This stability shows that our method minimizes the requirement for lengthy epochs and resource consumption while capturing structural and semantic changes early in training. Additionally, the inference procedure remains com-

putationally light and the process of downstream vulnerability predictions task is comparable fast and scalable, as the graph reasoning is precomputed alongside token embeddings rather than dynamically recalculated. To classify a single function from real-world usage, CodeVul+ requires on average 0.005 seconds on the GPU and about 0.05–0.1 seconds on a CPU (depending on the length of the function). We also measured end-to-end throughput in a batched setting: on a GPU, batching 64 functions yields about 1500 functions classified per second. Code parsing and disk I/O actually become bottlenecks before the model’s computation, indicating the value of multi-threaded data loaders to fully utilize GPU capacity. During inference with batch size 1, the memory footprint remains at about 1.2GB including frozen GCN and Transformer layers, making the model deployable even on mid-range GPUs. Besides, our framework is somewhat slower per epoch than using a codeptm like GraphCodeBERT fine-tuning. We estimate that CodeVul+ is about 30% slower per batch than a CodeBERT, due to computing graph embeddings and the handling of graph based resonance. Additionally, our approach operates at the function level and does not require whole-program analysis at inference, it can easily distribute across functions which indicates scalability to very large projects. In general, our architecture effectively balances computational efficiency and model expressiveness, making it suitable for deployment in real-world vulnerability detection pipelines.

H Cross Project Generalization

We validate our approach on diverse cases of refactoring and maintainability that reflect structural and semantic features of the software code that affect its long-term adaptability. From Table 5, our experimental results show impressive generalization despite being exclusively pretrained on C and C++ code. Our model obtains strong F1 scores in structurally related languages like C (0.686), Python (0.601), and PHP (0.575). Slightly lower performance in Ruby and Go indicates the limitations of pretrained alignment when faced with sparse or stylistically different languages.

I Qualitative Examples

To illustrate CodeVul+’s behavior, we provide examples of code snippets alongside the model’s predictions and an interpretation of what the model

784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815

816

817
818
819
820
821
822
823
824
825
826
827
828

829

830
831
832

learned. These cases help demonstrate how the model handles typical vulnerability patterns and why it makes certain predictions.

Example 1: Buffer Overflow Vulnerability.

Consider the C function below, which is intended to copy user input into a fixed-size buffer:

```
1 void copy_user_input(char *user_input) {
2     char buf[8];
3     strcpy(buf, user_input); // no bounds check
4     printf("Copied: %s\n", buf);
5 }
```

Prediction: Vulnerable (Confidence: 0.93)

With high confidence, CodeVul+ precisely detects this function as vulnerable, where the model assigns significant attention to line 3 due to the unsafe *strcpy* function being invoked without any bound check. This makes it vulnerable to buffer overflow when user *user_input* exceeds the buffer’s allotted size. However, the use of a statically allocated buffer (*char buf[8]*) and the lack of input validation are significant signs that fit typical vulnerability patterns. The GCN component’s role during training is to learn structural interconnections across these graphs, with the embedded token representations serving as input features. In this particular instance, the call for the *strcpy* function is given high attention and graph convolutional emphasis by the model and its relationship to the statically allocated buffer *buf[8]*.

Example 2: Floating-point Boolean Output.

```
1 static inline void gen_op_eval_fbo(
2     TCGv dst, TCGv src, unsigned int fcc_offset) {
3
4     gen_mov_reg_FCC0(dst, src, fcc_offset);
5     gen_mov_reg_FCC1(cpu_tmp0, src, fcc_offset);
6     tcg_gen_and_tl(dst, dst, cpu_tmp0);
7     tcg_gen_xori_tl(dst, dst, 0x1);
8 }
```

Prediction: Not Vulnerable (Confidence: 0.71).

Here, our suggested framework accurately determines that the following function is not vulnerable, where the key insight lies in the conditional structure that safeguards the *tcg_gen_and_tl* operation. Here, the GraphCodeBERT encoder concentrates on the semantic connection between the constants and bitwise operations, learning that the pattern reflects a regulated logical transformation rather than a misuse. Concurrently, our dynamically constructed graph of the code is transmitted to the GCN based reasoning, capturing the structural dependencies across the function. This approach grasps that *tcg_gen_xori_tl* is not arbitrarily invoked; rather, it follows a rigorous and uniform operand logic pattern across the computation graph. In contrast to

vulnerable patterns that exhibit erratic or unguarded data propagation, this snippet sample shows safe transformations. Additionally, the model picks up a high-confidence rule; operations like *xori* or *and* do not indicate risk if used in a logically consistent framework and conform to intermediate value checks.

Example 3: Use-After-Free:

```
1 void safe_copy(char *input) {
2     char buffer[64];
3     if (input != NULL && strlen(input) >> 2 < 16)
4     {
5         strncpy(buffer, input, sizeof(buffer) - 1);
6         buffer[sizeof(buffer) - 1] = '\0';
7     }
8 }
```

Prediction: Vulnerable (False Positive, confidence 0.83)

Our proposed embedding framework exhibits strong performance across conventional code patterns. However, misclassifications can occur in safety tests that are syntactically rare but semantically valid. In this case, the input length is restricted by a transformed value (*strlen(input) >> 2 < 16*), while it meets memory safety standards. However, deviation from the canonical pattern (*e.g., strlen(input) < sizeof(buffer)*) causes a lower level of confidence regarding the operation’s safety. This structural format reflects a broader challenge in learning generalized abstractions over semantic equivalence across diverse syntactic forms, which is a known drawback of numerous embedding frameworks. Importantly, these misclassifications are rare and usually are related to unusually defined or obfuscated safety criteria rather than actual vulnerability blind spots.

Through these examples, we see that CodeVul+ can precisely identify classic vulnerabilities and also refrain from false positives when safe coding practices are present. The model leverages both token-level cues and structural context to make decisions. However, low-confidence cases tend to occur on less frequent vulnerability types or when the vulnerable pattern spans across non-adjacent code which is consistent and known limitations in code representation learning. However, the qualitative behavior aligns with expectations and validates that the model has learned meaningful patterns akin to secure coding rules.

References

Akiko Aizawa. 2003. [An information-theoretic perspective of tf-idf measures](#). *Information Processing &*

939	Management, 39(1):45–65.	
940	Hajar Y Alimam, Wael A Mohamed, and Ayman S	
941	Selmy. 2024. Deep recurrent neural network ap-	
942	proach with lstm structure for hand movement recog-	
943	nition using emg signals. In <i>Proceedings of the 2023</i>	
944	<i>12th International Conference on Software and Infor-</i>	
945	<i>mation Engineering, ICSIE '23</i> , pages 58–65, New	
946	York, NY, USA. Association for Computing Machin-	
947	ery.	
948	Sercan O. Arik and Tomas Pfister. 2021. <i>Tabnet: At-</i>	
949	<i>tentive interpretable tabular learning</i> . <i>Proceedings</i>	
950	<i>of the AAAI Conference on Artificial Intelligence</i> ,	
951	35(8):6679–6687.	
952	Loubna Ben Allal, Raymond Li, Denis Kocetkov,	
953	Chenghao Mou, Christopher Akiki, Carlos	
954	Munoz Ferrandis, Niklas Muennighoff, Mayank	
955	Mishra, Alex Gu, Manan Dey, Logesh Kumar	
956	Umapathi, Carolyn Jane Anderson, Yangtian Zi, Joel	
957	Lamy Poirier, Hailey Schoelkopf, Sergey Troshin,	
958	Dmitry Abulkhanov, Manuel Romero, Michael	
959	Lappert, Francesco De Toni, Bernardo García del	
960	Río, Qian Liu, Shamik Bose, Urvashi Bhattacharyya,	
961	Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco	
962	Zocca, Sourab Mangrulkar, David Lansky, Huu	
963	Nguyen, Danish Contractor, Luis Villa, Jia Li,	
964	Dzmitry Bahdanau, Yacine Jernite, Sean Hughes,	
965	Daniel Fried, Arjun Guha, Harm de Vries, and	
966	Leandro von Werra. 2023. SantaCoder: Don't Reach	
967	for the Stars! <i>arXiv preprint arXiv:2312.06798</i> .	
968	Tim Boland and Paul E Black. 2012. Juliet 1. 1 c/c++	
969	and java test suite. <i>Computer</i> , 45(10):88–90.	
970	Yohan Brunebarbe. 2022. An algebraic approach to the	
971	hyperbolicity of moduli stacks of calabi-yau varieties.	
972	<i>Compositio Mathematica</i> , 158(5):1053–1078.	
973	Sicong Cao, Xiaobing Sun, Lili Bo, Ying Wei, and	
974	Bin Li. 2021. Bgmn4vd: Constructing bidirectional	
975	graph neural-network for vulnerability detection. <i>In-</i>	
976	<i>formation and Software Technology</i> , 136:106576.	
977	Available at: https://www.sciencedirect.com/	
978	science/article/pii/S0950584921000586 .	
979	Saikat Chakraborty, Rahul Krishna, Yangruibo Ding,	
980	and Baishakhi Ray. 2021. Deep learning based vul-	
981	nerability detection: Are we there yet? <i>IEEE Trans-</i>	
982	<i>actions on Software Engineering</i> , 48(9):3280–3296.	
983	Haochen Chen, Syed Fahad Sultan, Yingtao Tian,	
984	Muhao Chen, and Steven Skiena. 2022. Fast and	
985	accurate network embeddings via very sparse ran-	
986	dom projection. In <i>Proceedings of the 2022 Con-</i>	
987	<i>ference on Empirical Methods in Natural Language</i>	
988	<i>Processing (EMNLP)</i> . Association for Computational	
989	Linguistics.	
990	Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun	
991	Chen, and David Wagner. 2023. Diversevul: A	
992	new vulnerable source code dataset for deep learn-	
993	ing based vulnerability detection. In <i>Proceedings</i>	
994	<i>of the 26th International Symposium on Research in</i>	
995	<i>Attacks, Intrusions and Defenses</i> , pages 654–668.	
	Xiao Cheng, Guanqin Zhang, Haoyu Wang, and Yulei	996
	Sui. 2022. Path-sensitive code embedding via con-	997
	trastive learning for software vulnerability detection.	998
	In <i>Proceedings of the 31st ACM SIGSOFT Interna-</i>	999
	<i>tional Symposium on Software Testing and Analysis</i> ,	1000
	pages 519–531. ACM.	1001
	Davide Chicco and Giuseppe Jurman. 2022. The abc	1002
	recommendations for validation of supervised ma-	1003
	chine learning results in biomedical sciences. <i>Front-</i>	1004
	<i>iers in Big Data</i> , 5:979465.	1005
	Xiaohu Du, Ming Wen, Jiahao Zhu, Zifan Xie, Bin	1006
	Ji, Huijun Liu, Xuanhua Shi, and Hai Jin. 2024.	1007
	Generalization-enhanced code vulnerability detec-	1008
	tion via multi-task instruction fine-tuning. <i>arXiv</i>	1009
	<i>preprint arXiv:2406.03718</i> .	1010
	D. M. Endres and J. E. Schindelin. 2003. A new metric	1011
	for probability distributions. <i>IEEE Transactions on</i>	1012
	<i>Information Theory</i> , 49(7):1858–1860.	1013
	Jiahao Fan, Yi Li, Shaohua Wang, and Tien N Nguyen.	1014
	2020. Ac/c++ code vulnerability dataset with code	1015
	changes and cve summaries. In <i>Proceedings of the</i>	1016
	<i>17th international conference on mining software</i>	1017
	<i>repositories</i> , pages 508–512.	1018
	Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xi-	1019
	aocheng Feng, Ming Gong, Linjun Shou, Bing Qin,	1020
	Ting Liu, Daxin Jiang, et al. 2020. Codebert: A	1021
	pre-trained model for programming and natural lan-	1022
	guages. <i>ArXiv preprint arXiv:2002.08155</i> .	1023
	Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and	1024
	Artem Babenko. 2021. Revisiting deep learning mod-	1025
	els for tabular data. <i>Advances in Neural Information</i>	1026
	<i>Processing Systems (NeurIPS)</i> , 34:18932–18943.	1027
	Allan Grönlund, Lior Kamma, and Kasper Green	1028
	Larsen. 2020. Near-tight margin-based generaliza-	1029
	tion bounds for support vector machines. In <i>Proceed-</i>	1030
	<i>ings of the 37th International Conference on Machine</i>	1031
	<i>Learning</i> , volume 119 of <i>Proceedings of Machine</i>	1032
	<i>Learning Research</i> , pages 3779–3788. PMLR.	1033
	Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming	1034
	Zhou, and Jian Yin. 2022. Unixcoder: Unified cross-	1035
	modal pre-training for code representation. <i>arXiv</i>	1036
	<i>preprint arXiv:2203.03850</i> .	1037
	Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu	1038
	Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svy-	1039
	atkovskiy, Shengyu Fu, Michele Tufano, Shao Kun	1040
	Deng, Colin Clement, Dawn Drain, Neel Sundaresan,	1041
	Jian Yin, Daxin Jiang, and Ming Zhou. 2021. Graph-	1042
	codebert: Pre-training code representations with data	1043
	flow. In <i>Proceedings of the 2021 International Con-</i>	1044
	<i>ference on Learning Representations (ICLR)</i> .	1045
	Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar	1046
	Karnin. 2020. Tabtransformer: Tabular data mod-	1047
	eling using contextual embeddings. In <i>Proceedings</i>	1048
	<i>of the 26th ACM SIGKDD International Conference</i>	1049
	<i>on Knowledge Discovery and Data Mining</i> , pages	1050
	1243–1253. ACM.	1051

- J. Ingram, C. J. Hand, and G. Maciejewski. 2016. [Exploring the measurement of markedness and its relationship with other linguistic variables](#). *PLoS One*, 11(6):e0157141. 1106
- Thomas K. Landauer, Peter W. Foltz, and Darrell Laham. 1998. [An introduction to latent semantic analysis](#). *Discourse Processes*, 25(2-3):259–284. 1107
- Yi Li, Shaohua Wang, and Tien N. Nguyen. 2023a. Vulnerability detection with fine-grained interpretations. *ArXiv preprint*. 1108
- Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2023b. Sysevr: A framework for using deep learning to detect software vulnerabilities. *ArXiv preprint*. 1109
- Shuxiao Ma, Wei Huang, Nana Cui, Zhaoyang Cai, Yan Xu, and Yue Qiao. 2023. [Exploring non-linear and synergistic effects of street environment on the spirit of place in historic districts: Using multi-source data and xgboost](#). *Sustainability*, 15(2):1456. 1110
- Han Cheol Moon, Tasnim Mohiuddin, Shafiq Joty, and Chi Xu. 2021. [A unified neural coherence model](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Online. Association for Computational Linguistics. 1111
- Hoang H. Nguyen, Nhat-Minh Nguyen, Chunyao Xie, Zahra Ahmadi, Daniel Kudendo, Thanh-Nam Doan, and Lingxiao Jiang. 2023. Mando: Multi-level heterogeneous graph embeddings for fine-grained detection of smart contract vulnerabilities. *ArXiv preprint*. 1112
- OpenAI. 2023. Chatgpt: language model (mar 14 version). <https://openai.com/chatgpt>. Accessed: 2025-05-13. 1113
- Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. [Glove: Global vectors for word representation](#). In *EMNLP*, pages 1532–1543. 1114
- Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. [Deepwalk: Online learning of social representations](#). In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 701–710, New York, NY, USA. ACM. 1115
- Jonathan Pisane, Raphaël Marée, Philippe Ries, Louis Wehenkel, and Jacques Verly. 2009. [An extra-trees-based automatic target recognition algorithm](#). In *Proceedings of the International Conference on Image Processing*, pages 1–6. IEEE. 1116
- Rebecca L. Russell, Louis Kim, Lei H. Hamilton, Tomo Lazovich, Jacob A. Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc W. McConley. 2020. [Automated vulnerability detection in source code using deep representation learning](#). *arXiv preprint arXiv:2003.07822*. 1117
- A.I. Schein and L.H. Ungar. 2007. [Active learning for logistic regression: an evaluation](#). *Machine Learning*, 68(3):235–265. 1118
- Gowthami Somepalli, Micah Goldblum, Avi Schwarzschild, C. Bayan Bruss, and Tom Goldstein. 2021. [SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training](#). *arXiv preprint arXiv:2106.01342*. 1119
- Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. [Autoint: Automatic feature interaction learning via self-attentive neural networks](#). In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1161–1170. ACM. 1120
- Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *ArXiv preprint arXiv:2109.00859*. 1121
- Dan Yu, Mohammadhussen Rafieisakhaei, and Suman Chakravorty. 2021. [A decoupled data based approach to stochastic optimal control problems](#). *Mathematics of Operations Research*, 46(1):39–63. 1122
- Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in neural information processing systems*, 32. 1123
- Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2020. [vuldeepecker: A deep learning-based system for multiclass vulnerability detection](#). In *Proceedings of the 2020 IEEE/ACM 42nd International Conference on Software Engineering*, pages 1105–1117. IEEE. 1124